

MIPSolvers 原生内点法 (Native IPM) 求解器

数学模型与最严苛代码审计报告

代码审计 (以实现为准, 不以注释为准)

2026 年 8 月 1 日

摘要

本报告对 MIPSolvers 仓库中原生内点法求解器族 (NLP-IPM、LP-IPM、LCQP-IPM、锥 IPM 及其 KKT/线性代数支撑层) 做两件事: (i) 以**代码实际行为**为准, 系统重建其数学模型; (ii) 从通用 NLP 求解器 (Ipopt / Wächter–Biegler 框架) 与主流 LP/锥 IPM 工程标准出发, 对数据结构与算法效率做最严苛评估, 并逐条登记占位符、虚假实现与死代码。审计结论是双面的: **锥 IPM 与 LP-IPM 内核大体真实且工程质量不低; 而 NLP-IPM (ipm_solver.cpp) 存在若干影响收敛语义与结果真实性的严重问题**——包括可将仅原始可行、对偶不可行的点报告为“收敛”的容差后门、被旁路的 filter 线搜索、冻结的有限差分 Hessian 伴随每迭代死计算、以及失败时静默委托外部 Ipopt 的回退链; 另有一批声明了却从未接线的选项 (`use_inertia_correction`、`crossover`、`max_correctors`、`presolve` 等) 与文档虚标 (LCQP 自称 Mehrotra 预测-校正, 实为单步中心法)。所有实质性结论均给出文件: 行号证据。

目录

1	审计范围、方法与证据标准	2
1.1	范围	2
1.2	方法与证据标准	2
1.3	模块地图	2
2	数学模型 (以代码实际实现为准)	3
2.1	NLP-IPM: 问题形式与内部不等式化	3
2.2	障碍子问题与 KKT 残差	3
2.3	Newton 系统: 凝聚与增广两种形式	4
2.4	Hessian 的来源与真实含义	4
2.5	Filter 线搜索 (Wächter–Biegler 框架)	4
2.6	μ 调度与内外层结构	5
2.7	Merit 驱动 (备选/回退路径)	5
2.8	可行性恢复 (restoration)	5
2.9	缩放	6
2.10	LP-IPM (NativeIPMLPAdapter)	6
2.11	LCQP-IPM (NativeLCQPAdapter)	6

目录	2
2.12 锥 IPM (ConicIPMSolver)	6
3 与 Ipopt (Wächter–Biegler 2006) 逐组件对照	7
4 占位符、虚假实现与死代码清单	8
4.1 严重 (影响结果正确性/真实性)	8
4.2 中等 (语义偏离/鲁棒性)	10
4.3 轻微	10
5 数据结构与算法效率评估 (通用 NLP 求解器视角)	10
5.1 每迭代成本链 (Filter 驱动, 默认路径)	10
5.2 结构性 red flags	11
5.3 各内核效率小结	12
6 测试覆盖与文档一致性	12
7 结论与整改优先级	12
A 选项接线普查表	13

1 审计范围、方法与证据标准

1.1 范围

本次审计覆盖 `src/engine/kernel/ipm/`、`src/engine/kernel/kkt/`、`src/engine/kernel/linear_algebra/` (除 HiGHS 引入的 `highs_factor/` 外) 的全部源文件与对应头文件, 以及 `src/engine/api/solver.cpp`、`src/engine/strategy/dispatcher.cpp` 中的适配器注册/分派链、`tests/test_ipm_solver.cpp`、`tests/test_conic_ipm.cpp` 与 `docs/` 内相关声明。B&C(MILP) 侧不在本次范围内; 但 `bc/parallel/`、`bc/root/`、`bc/search/` 三个目录为空 (仅有目录结构、零源文件) 这一事实顺带记录, 供后续 MILP 审计参考。

1.2 方法与证据标准

1. **不信任注释。**任何“声称实现了 X”的注释/docstring 只有在代码中定位到对应控制流才被接受; 注释与实现冲突时以实现为准, 并将该注释登记为“虚标”。
2. **全文件逐行通读核心模块:** `ipm_solver.cpp` (2624 行)、`kkt_system.cpp/kkt_system.hpp` (1105 行)、`linear_solver.cpp/.hpp`、`cholmod_ldlt.cpp`、`ipm_filter.cpp`、`ipm_restoration.cpp`、`ipm_scaling.cpp`、`lcqp_solver.cpp` (693 行)、`cones.cpp` (547 行)、`conic_ipm_solver.cpp` 主循环与 KKT 后端、`ipm_lp_solver.cpp` 主迭代段 (含预测/校正、收敛判据、步长)。
3. **接线普查:** 对 `IPMOptions/IPMLPOptions/NLPSolverOptions` 的每个字段全库 grep, 区分“真实接线 (影响控制流) / 只读不用 / 未接线”。
4. 每条实质性指控给出 文件: 行号; 个别难以 100% 确证之处明确标注“不确定”及已核查的程度。

1.3 模块地图

文件	行数	角色
<code>src/engine/kernel/ipm/ipm_solver.cpp</code>	2624	NLP-IPM 双驱动 (Filter/Merit) + 回退链
<code>src/engine/kernel/ipm/ipm_filter.cpp</code>	56	Wächter-Biegler filter 数据结构
<code>src/engine/kernel/ipm/ipm_restoration.cpp</code>	248	可行性恢复 NLP 构建
<code>src/engine/kernel/ipm/ipm_scaling.cpp</code>	200	梯度行缩放 (无列缩放)
<code>src/engine/kernel/ipm/ipm_lp_solver.cpp</code>	3497	LP 专用 Mehrotra IPM + 节点缓存
<code>src/engine/kernel/ipm/lcqp_solver.cpp</code>	693	界约束 QP 原始-对偶 IPM
<code>src/engine/kernel/ipm/conic_ipm_solver.cpp</code>	1159	锥 IPM (cvxopt 标准型, NT 缩放)
<code>src/engine/kernel/ipm/cones.cpp</code>	547	锥代数: svec/SOC Jordan/NT 缩放/步长
<code>src/engine/kernel/ipm/chordal_decomposition.cpp</code>	432	稀疏 SDP 弦分解 (团树骨架)
<code>src/engine/kernel/kkt/kkt_system.cpp/.hpp</code>	1105	KKT 组装、惯性“认证”、回代
<code>src/engine/kernel/linear_algebra/linear_solver.cpp</code>	423	分解后端分派 (LU/KLU/UMFPACK/MUMPS)
<code>src/engine/kernel/linear_algebra/cholmod_ldlt.cpp</code>	184	CHOLMOD SPD 封装 (锥 KKT 用)

适配器注册(`src/engine/api/solver.cpp:162--166`): `NativeIPMLPAdapter(LP)`、`NativeIPMAdapter` (NLP, 即 `NativeIPM`)、`NativeNLPAdapter` 等。NLP 分派顺序为 `Ipopt` → `NativeIPM` → `Na-`

tiveNLP (src/engine/strategy/dispatcher.cpp:77), 即 **native IPM** 不是 NLP 的默认首选, 外部 Ipopt 在前。

2 数学模型（以代码实际实现为准）

本章每个公式都给出实现锚点。记号: n 为变量数, $m_{\text{eq}} = |g|$, $m_{\text{nl}} = |h|$ 。

2.1 NLP-IPM: 问题形式与内部不等式化

求解器接受 NLPModel (problem_types.hpp:235):

$$\min_x f(x) \quad \text{s.t.} \quad g(x) = 0, \quad h(x) \leq 0, \quad \ell \leq x \leq u, \quad (1)$$

通过回调 `f, grad, hess, g, jac_g, h, jac_h` 取值; `sense=Maximize` 时在梯度/Hessian 上取负 (ipm_solver.cpp:61--63, 301--303)。

界约束不单独处理, 而是拆成两类线性不等式并入 h : 对每个有限下界列生成行 $\ell_j - x_j \leq 0$ (Jacobian 元 -1), 每个有限上界列生成行 $x_j - u_j \leq 0$ ($+1$) (证据: ipm_solver.cpp:420--430; 同构实现 kkt_system.cpp:816--871)。合并后不等式维数

$$m = m_{\text{nl}} + |\text{lb_cols}| + |\text{ub_cols}|, \quad (2)$$

引入松弛 $s > 0$ 与乘子 $\mu > 0$, 约束变为 $h(x) + s = 0$ 。**评注**: 主流 NLP 求解器 (Ipopt) 对界使用专属乘子 z^L, z^U 与对角结构, 不扩大 m ; 此处对全界问题不等式数直接加 $2n$, 增广 KKT 维数随之增加 (见 §5)。

2.2 障碍子问题与 KKT 残差

障碍参数记 $\bar{\mu}$ (初值 $\max(\mu_{\min}^{\text{eff}}, \text{mu_init}=0.1)$, ipm_solver.cpp:1042--1044)。障碍子问题的价值函数

$$\varphi_{\bar{\mu}}(x, s) = f(x) - \bar{\mu} \sum_{i=1}^m \log s_i \quad \text{ipm_solver.cpp:971--979} \quad (3)$$

KKT 残差 (绝对尺度):

$$r_d = \nabla f(x) + J_g^\top \lambda + J_h^\top \mu, \quad r_{\text{eq}} = g(x), \quad r_{\text{ineq}} = h(x) + s, \quad r_{\text{comp}} = s \circ \mu - \bar{\mu} \mathbf{1}, \quad (4)$$

(证据: ipm_solver.cpp:1163--1170)。外层收敛判据为**绝对残差**

$$\max(\|r_{\text{eq}}\|_\infty, \|r_{\text{ineq}}\|_\infty) \leq \text{tol_primal}, \quad \|r_d\|_\infty \leq \text{tol_dual}, \quad \|s \circ \mu\|_\infty \leq \text{tol_complementarity}, \quad (5)$$

默认均为 10^{-6} (证据: ipm_solver.cpp:1195--1200, 默认见 ipm_solver.hpp:25--28)。代码另算一个缩放 merit $\max$ (缩放原始/对偶/互补) 仅用于 **best-iterate** 排序与线搜索, 不用于终止 (证据: ipm_solver.cpp:930--954) ——即没有 Ipopt 式 s_D, s_C 尺度化终止判据。

2.3 Newton 系统：凝聚与增广两种形式

凝聚路径（默认）。消去 $ds, d\mu$ 后求解

$$\underbrace{\begin{bmatrix} W & J_g^\top \\ J_g & 0 \end{bmatrix}}_K \begin{bmatrix} dx \\ d\lambda \end{bmatrix} = \begin{bmatrix} r_x \\ -r_{eq} \end{bmatrix}, \quad W = H + J_h^\top S^{-1} M J_h, \quad (6)$$

其中 $S = \text{diag}(s)$, $M = \text{diag}(\mu)$, H 为 Lagrangian Hessian (§2.4), 右端

$$r_x = -r_d - J_h^\top S^{-1} (M r_{ineq} + \bar{\mu} \mathbf{1} - S\mu), \quad (7)$$

方向恢复 $ds = -r_{ineq} - J_h dx$, $d\mu = S^{-1}(\bar{\mu} \mathbf{1} - S\mu - M ds)$ (证据: 组装 `ipm_solver.cpp:1260--1267`, 右端 1286–1297, 恢复 1343–1349)。 $J_h^\top (M/S) J_h$ 通过逐行缩放后稀疏转置乘得到 (证据: `ipm_solver.cpp:458--468, 1262--1266`)。

增广路径 (`use_augmented_newton`, 默认 `false`) 求解未凝聚系统

$$\begin{bmatrix} H + \delta_W I & J_g^\top & J_h^\top \\ J_g & -\delta_C I & 0 \\ J_h & 0 & -SM^{-1} \end{bmatrix} \begin{bmatrix} dx \\ d\lambda \\ d\mu \end{bmatrix} = \begin{bmatrix} -r_d \\ -r_{eq} \\ -r_{ineq} - M^{-1}(\bar{\mu} \mathbf{1} - S\mu) \end{bmatrix}, \quad (8)$$

并附 ≤ 2 步迭代精化 (证据: 组装 `ipm_solver.cpp:623--657`, 右端 1232–1238, 精化 661–683)。两条路径都使用“结构指纹 + scatter map”的 `analyze-once/fill- $O(\text{nnz})$` 装配缓存 (证据: `ipm_solver.cpp:498--657`; `kkt_system.cpp:93--198`) ——这是本代码库中工程水准较高的部分。

2.4 Hessian 的来源与真实含义

`build_lagrangian_hessian` (`ipm_solver.cpp:279--315`) 按优先级取:

1. `lagrangian_hess` 回调: $\nabla^2 f + \sum_i \lambda_i \nabla^2 g_i + \sum_{i \leq m_{nl}} \mu_i \nabla^2 h_i$ (唯一含约束曲率的来源);
2. 否则 `hess` 回调: 仅 $\nabla^2 f$ (约束曲率静默缺失, 无任何检查或警告) (证据: `ipm_solver.cpp:294--303`);
3. 否则拟牛顿态: 若已有 `fd_hess` 则永远取它, 否则取对角 + 低秩块矩阵, 最后退化为 I (证据: `ipm_solver.cpp:304--313`)。

其中 `fd_hess` 是在初始点 x_0 对 `grad` 做逐列有限差分 (n 次梯度评估, $n \leq 2000$ 门控) 得到的目标函数 Hessian, 此后在 Filter 路径全程冻结不再刷新 (证据: FD 构建 `ipm_solver.cpp:186--228`; 启动时一次性刷新 1079–1082; Filter 路径仅在 `has_fd_hess` 为假时刷新 1136–1141)。该设计的后果见 §4–5。

2.5 Filter 线搜索 (Wächter–Biegler 框架)

违反度与方向导数

$$\theta(x, s) = \|g(x)\|_1 + \|h(x) + s\|_1, \quad \nabla \varphi_{\bar{\mu}}^\top d = \nabla f^\top dx - \bar{\mu} \sum_i \frac{ds_i}{s_i}, \quad (9)$$

(证据: `ipm_solver.cpp:960--992`)。filter 本身的接受语义 (对已有条目施加 $(1-\gamma_\theta)\theta_k, \varphi_k - \gamma_\varphi\theta_k$ 边际, 禁止域判定、支配剪枝、 θ 上界初始化) 与 Wächter–Biegler 一致, 实现正确 (证据: `ipm_filter.cpp:9--52`)。接受逻辑 (`ipm_solver.cpp:1490--1512`):

- 切换条件（switching）与 Armijo: $\varphi^{try} \leq \varphi_k + \eta_\varphi \alpha \text{ slope (f-type)}$;
- 否则 h-type: $\theta^{try} \leq (1 - \gamma_\theta)\theta_k$ 或 $\varphi^{try} \leq \varphi_k - \gamma_\varphi \theta_k$;
- 非 f-type 接受后将 (θ_k, φ_k) 加入 filter（证据: ipm_solver.cpp:1712--1716）;
- 二阶修正（SOC）: 首次拒绝且 $\theta^{try} > \theta_k$ 时以 $c_{soc} = \alpha r_{eq} + g(x + \alpha dx)$ 重解同一分解一次, 仅一次（证据: ipm_solver.cpp:1564--1668）。

但与论文存在四处**实质偏离**（详见 §4）: (i) 存在可绕过 filter 的 direct-accept 后门; (ii) 切换条件含恒真子句; (iii) SOC 仅在 $m_{eq} > 0$ 时触发; (iv) 对偶步长随原始 α 同步收缩（Ipopt 中 α_{dual} 在回退中固定）。另: μ 下调时 `filter.clear()`（证据: ipm_solver.cpp:1731）。

2.6 $\bar{\mu}$ 调度与内外层结构

Fiacco–McCormick 单调框架: 内层收敛 $E_{\bar{\mu}} = \max(\|r_d\|_\infty, \|r_{eq}\|_\infty, \|r_{ineq}\|_\infty, \|r_{comp}\|_\infty) \leq \kappa_\varepsilon \bar{\mu}$ ($\kappa_\varepsilon=10$) 后,

$$\bar{\mu} \leftarrow \max(\mu_{\min}^{\text{eff}}, \min(\kappa_\mu \bar{\mu}, \bar{\mu}^{\theta_\mu})), \quad \kappa_\mu=0.2, \theta_\mu=1.5, \quad (10)$$

外层上限 60 次, 总迭代上限 `max_iter=400`（证据: ipm_solver.cpp:1092--1093, 1203--1208, 1721--1731）。没有 Ipopt 的自适应 μ （probing/quality-function）策略。

2.7 Merit 驱动（备选/回退路径）

globalization=Merit 或 Filter 失败后的回退驱动（ipm_solver.cpp:2115--2622）: Mehrotra 式预测-校正——仿射方向（ $\sigma=0$ ）+ 逐分量中心化目标

$$\sigma_i = \text{clamp}\left(\left(\frac{(s + \alpha_p^{aff} ds^{aff})_i (\mu + \alpha_d^{aff} d\mu^{aff})_i}{\max(s_i \mu_i, 10^{-12})}\right)^3, 0, 1\right), \quad \text{target}_i = \sigma_i s_i \mu_i, \quad (11)$$

校正右端含二阶项 $ds^{aff} \circ d\mu^{aff}$, 另有 ≤ 2 次 Gondzio 式附加修正（ $\beta=0.1$ ）（证据: 目标函数 ipm_solver.cpp:317--341; 主流程 2306--2431）。线搜索为自造准则 `sufficient_primal_dual_progress`（缩放残差充分下降或四项中两项改善, **非** Armijo/penalty merit）（证据: ipm_solver.cpp:886--911, 2464--2475）; 停滞 6 次则重置松弛/乘子（recenter）（证据: 2290--2294）。**该路径无 restoration、无 $\bar{\mu}$ 调度、无 inertia 处理**, KKT 用盲 Tikhonov 正则（ $10^{-9} \rightarrow 10^{-2}$ 十倍递增）（证据: ipm_solver.cpp:470--480）。

2.8 可行性恢复（restoration）

触发于 Filter 失败且 `use_restoration_phase`（默认开）且 `prob.g` 存在（仅等式问题）（证据: ipm_solver.cpp:1877--1931）。构建恢复 NLP

$$\min_{x,p,n} \sum_{i=1}^{m_{eq}} (p_i + n_i) + \frac{\zeta}{2} \|D_R(x - x_R)\|^2 \quad \text{s.t.} \quad g(x) + p - n = 0, \quad h(x) \leq 0, \quad p, n \geq 0, \quad (12)$$

$\zeta = 10^{-4}$, $D_R = \text{diag}(\min(1, 1/\max(1, |x_{R,i}|)))$ （证据: ipm_restoration.cpp:10--17, 66--192）。与 Wächter–Biegler §3.3 形式一致（但只松弛等式）; 解出 x_{new} 后**从头重解原问题**（一次性），而非在同一运行内继续（证据: ipm_solver.cpp:1900--1929）。

2.9 缩放

梯度行缩放: $s_f = g_{\max}/\|\nabla f(x_0)\|_\infty$ (仅当超过 $g_{\max}=100$), $s_{g,i}, s_{h,i}$ 取 Jacobian 行 ∞ 范数同理; 在 x_0 计算一次; **无列(变量)缩放** (证据: `ipm_scaling.cpp:31--77`)。缩放模型通过 `lambda` 包装器透明还原乘子 ($\lambda_{orig} = (s_g/s_f)\lambda_s$), 实现正确 (证据: `ipm_scaling.cpp:79--198`)。

2.10 LP-IPM (NativeIPMLPAdapter)

问题 $\min c^\top x$ s.t. $Ax \leq b$, $A_{eq}x = b_{eq}$, $\ell \leq x \leq u$; 内部转标准形 (不等式加松弛并入 A_e), Ruiz 均衡 (默认 10 轮, 失败重试 0/4 $\times$ 轮) (证据: `ipm_lp_solver.cpp:305--368, 395--405`)。原始-对偶系统:

$$r_p = b - A_e x, \quad r_d = c - A_e^\top y - z_l + z_u, \quad \mu = \frac{1}{n_{compl}} \sum_j (g_j^l z_j^l + g_j^u z_j^u), \quad (13)$$

其中 $g^l = x - \ell$, $g^u = u - x$ 为界间隙 (证据: `ipm_lp_solver.cpp:1717--1735`)。每迭代构造 $\Theta = \text{diag}(z/g + \text{reg})$, 解正规方程 $N = A_e \Theta A_e^\top$ (稀疏/带状/稠密三后端, 含稠密列防护) 或增广 KKT (稠密耦合模型) (证据: 后端分派注释与实现 `ipm_lp_solver.cpp:723--750, 1557--1701`)。方向为**教科书 Mehrotra 预测-校正**:

$$\sigma = \min((\mu_{aff}/\mu)^3, 0.5), \quad \text{校正右端} = \text{中心化} + ds^{aff} \circ dz^{aff} \text{ 二阶项}, \quad (14)$$

(证据: 预测 1885–1920, σ 1922–1934, 校正 1952–1990); 步长为 $\tau=0.9995$ 的 fraction-to-boundary (证据: `ipm_lp_solver.cpp:90, 1900--1920`); 正则 $\text{reg} = \text{clamp}(10^{-6}\mu, \text{floor}, 10^{-2})$ (证据: 1760)。收敛判据 (绝对) $\text{pfeas} < 10^{-8}$, $\text{dfeas} < 10^{-8}$, $\mu < 10^{-8}$ (证据: 1740–1748)。工程亮点: `prepare_for_node_solves/solve_cached_node` 为 B&C 节点 LP 提供结构缓存 (CSR, scatter map, 符号分解复用) (证据: `ipm_lp_solver.cpp:2120--3497`)。死选项见 §4)。

2.11 LCQP-IPM (NativeLCQPAdapter)

问题 $\min \frac{1}{2}x^\top Qx + c^\top x$ s.t. $A_{eq}x = b_{eq}$, $\ell \leq x \leq u$ (一般不等式 $Ax \leq b$ 经松弛转等式后递归)。原始-对偶 KKT:

$$\begin{bmatrix} Q_s + D + \rho I & A_{eq,s}^\top \\ A_{eq,s} & -\rho I \end{bmatrix} \begin{bmatrix} dx \\ dy \end{bmatrix} = - \begin{bmatrix} r'_{dual} \\ r_{eq} \end{bmatrix}, \quad D = z_{lb}/s_{lb} + z_{ub}/s_{ub}, \quad (15)$$

$\rho=10^{-8}$ 固定; 符号分解一次、每迭代仅更新对角 (证据: `lcqp_solver.cpp:413--453, 593--613`)。Ruiz 均衡真实实现 (证据: 49–97)。但方向不是 **Mehrotra 预测-校正**: 每迭代仅一次中心步, 中心参数 $\sigma = \min(0.3, 100\mu)$, 无仿射方向、无 μ_{aff} 、无二阶修正 (证据: `lcqp_solver.cpp:615--680`) —— 与文件头 “Mehrotra Predictor-Corrector” (第 377 行注释) 直接矛盾 (见 §4)。收敛判据之外另有 “Optimal (near)” 与 “stall-accept” 两个**不要求对偶可行**的收敛出口 (证据: 533–544, 567–579)。

2.12 锥 IPM (ConicIPMSolver)

cvxopt 标准型

$$(P) \min c^\top x \text{ s.t. } Gx + s = h, Ax = b, s \in \mathcal{K}; \quad (D) \max -h^\top z - b^\top y \text{ s.t. } G^\top z + A^\top y + c = 0, z \in \mathcal{K}, \quad (16)$$

$\mathcal{K} = \mathbb{R}_+^l \times \prod_i \mathcal{Q}^{q_i} \times \prod_j \mathcal{S}_+^{s_j}$ (svec 约定) (证据: `problem_types.hpp:176--213`)。每迭代计算 Nesterov–Todd 缩放: 象限 $d = \sqrt{s/z}$; SOC 用 $(\bar{s}, \bar{z}) \rightarrow (\bar{w}, \gamma, \beta)$ 闭式与 $v = \bar{w}^{1/2}$ (Jordan 平方根) (证据: `cones.cpp:341--361`); SDP 用 $L_z^\top L_s$ 的 SVD 构造 R 因子 (证据: `cones.cpp:370--402`)。方向为 CVXOPT `conelp` 式: 先仿射 ($r_l = -\lambda$), $\sigma = \text{clamp}((\mu_{aff}/\mu)^3, 0, 1)$, 再组合 (校正 + 中心化) 方向, Schur KKT $K = [G^\top H^{-1}G + \delta I, A^\top; A, -\delta I]$ 以 CHOLMOD (SPD) 分解、失败回退 MUMPS 对称不定 (证据: `conic_ipm_solver.cpp:602--724, 1102--1132`)。步长取 λ 空间锥边界步长的 0.99 倍 (证据: 59,1135–1136)。终止: 相对尺度残差 $\leq \text{feastol}$ 且 ($\text{gap} \leq \text{abstol}$ 或 $\text{relgap} \leq \text{reltol}$); 另有**真实的**原始/对偶不可行证书与 3×3 KKT 迭代精化 (证据: 976–1032, 1074–1098)。稀疏 SDP 可走弦分解 (分解 $\rightarrow$ 求解 $\rightarrow$ 对偶恢复 $\rightarrow$ 原始残差再验证) (证据: 866–932; `chordal_decomposition.cpp` 含最小度式消去序与极大团过滤骨架, 本报告未逐行验证其数值正确性——标注为“不确定”)。该模块与 `docs/conic_sdp.md` 的设计文档一致性高。

3 与 Ipopt (Wächter–Biegler 2006) 逐组件对照

组件	Ipopt 标准	本实现 (判定)
障碍子问题/原始-对偶 Newton filter 数据结构	一致	一致 (凝聚与增广两式均正确) [真实]
switching 条件	WB (2.9)–(2.10)	语义正确 [真实] (<code>ipm_filter.cpp</code>)
	WB (2.11), 含 $\theta \leq \theta_{\min}$ 门控	含恒真子句, 门控失效 [偏离] (<code>ipm_solver.cpp:1494--1498</code>)
Armijo (f-type) / h-type	WB (2.12)/(2.13)	实现一致, 但可被 direct-accept 旁路 [偏离] (1514–1523)
二阶修正 SOC	对等式 + 不等式	仅 $m_{eq} > 0$ 触发 [部分] (1564–1565)
可行性恢复	松弛全部约束, 同运行内继续	仅等式; 另起完整重解 [部分] (1877–1931)
μ 策略	自适应 (probing) + 单调兜底	仅单调 [简化] (1721–1731)
inertia 校正	分解惯性计数 (MA57 INFOG) $\Rightarrow \delta_W$	无分解惯性; $Z^\top W Z$ 稠密证书 ($\text{nul-lity} \leq 128$) 或全空间 SimplicialLDLT [偏离] (<code>kkt_system.cpp:562--772</code>)
KKT 后端	MA27/57/86/97, Pardiso (对称不定 LDL $^\top$)	默认非对称 LU (KLU/UMFPACK/EigenSparseLU); MUMPS 存在但非默认 [偏离] (<code>linear_solver.cpp:404--421</code>)
线搜索导数评估	每试点仅 f 与约束值	每试点重算全部 Jacobian [低效] (<code>ipm_solver.cpp:849--884</code>)
尺度化终止判据	s_D, s_c 尺度化 KKT 误差	绝对残差; 缩放仅用于排序 [偏离] (930–954, 1195–1200)
acceptable 级别	对偶 + 原始 + 互补均需 acceptable	仅原始 $\leq 10^{-2}$ 即收敛、对偶置零 [严重偏离] (2513–2545)
不可行证书	restoration 收敛即判不可行迹象	无 [缺失]
导数检查器	有 (derivative_checker)	无 (全库 grep 无对应实现) [缺失]
列缩放	有	无 (仅行缩放) [缺失] (<code>ipm_scaling.cpp</code>)
warm start	有 (含固定变量处理)	有 (三向量暖启动 + Ipopt 态精化) [真实] (1052–1066, 2035–2052)

4 占位符、虚假实现与死代码清单

按严重程度分级。每条给出证据与影响。

4.1 严重（影响结果正确性/真实性）

P1. “仅原始可行即收敛”的容差后门（Merit 路径） 当严格容差未达且 `tol_accept>0`（默认 10^{-2} ）时，只要 best iterate 的原始残差 $\leq 10^{-2}$ 即置 `converged=true`、状态 “Converged (acceptable tolerance)”，并且对偶全部置零输出（证据：ipm_solver.cpp:2513--2545，注释自承 “dual infeasibility is tolerated”）。对照 Ipopt: acceptable 级别要求原始、对偶、互补同时低于 acceptable 容差。由于默认 Filter 失败链会落入 Merit 驱动（2001–2012），默认配置下此后门可达：一个仅近似可行、离最优任意远的点可被报告为 “收敛”。

P2. 冻结的有限差分 Hessian + 每迭代 QN 死计算 无 Hessian 回调且 $n \leq 2000$ 时：启动期花 n 次梯度评估构建 x_0 处 FD Hessian (仅目标曲率)；之后每迭代执行 `update_quasi_newton_state` 更新对角 + 低秩块，但 `build_lagrangian_hessian` 因 `has_fd_hess` 恒真而永远丢弃这些更新，取冻结 FD 矩阵（证据：更新调用 ipm_solver.cpp:1136--1141；选择逻辑 304–313）。后果双料：(i) 所谓 “Newton 法” 对无 Hessian 的问题实为固定初始 Hessian 的不精确法，且不含任何约束曲率；(ii) 每迭代的 QN 更新是纯死计算 ($O(n)$ 以上开销)。仅当 $n > 2000$ (FD 被门控) 时，对角 + 块 QN 才真正进入 KKT。

P3. 约束曲率静默缺失 用户只提供 `hess`（目标 Hessian）而未提供 `lagrangian_hess` 时， W 直接取 $\nabla^2 f$ ，无 “约束是否线性” 检查，无任何警告；对非线性约束问题这不再是牛顿法，收敛性质改变且用户无从知晓（证据：ipm_solver.cpp:294--303）。

P4. filter 被 direct-accept 后门旁路 常规接受之外，`sufficient_primal_dual_progress`（自造准则）与 `aggressive_direct` ($\alpha \geq 0.8\alpha_0$ 且缩放 merit 降 5%) 可在 `filter_ok=false` 时接受试点（证据：ipm_solver.cpp:1514--1523；SOC 分支同构 1646–1653）。这破坏了 filter 方法的全局收敛构造（接受 filter 明确禁止的点），“Wächter–Biegler filter line search” 的名实不符——实际是一个 filter/merit 混合启发。

P5. switching 条件含恒真子句（死代码 + 语义改变）

```
const bool switching =
    (slope_k < 0.0) &&
    (alpha * std::pow(-slope_k, opt.filter_s_phi) >
     opt.filter_delta * std::pow(theta_k, opt.filter_s_theta)) &&
    (theta_k <= theta_min || opt.filter_s_phi > 0.0);
```

（证据：ipm_solver.cpp:1494--1498）。默认 `filter_s_phi=2.3 > 0`，第三子句恒真： $\theta_{\min}$ 门控失效（`filter_theta_min_scale` 成为只读不用选项），且这是典型的 “看起来严谨实则空转” 代码。

P6. LCQP 自称 Mehrotra 预测-校正，实为单步中心法 文件头与迭代注释均称 “Primal-Dual IPM with Mehrotra Predictor-Corrector”，实现为每迭代一次中心步 $\sigma = \min(0.3, 100\mu)$ ，无预测器/校正器结构（证据：注释 `lcqp_solver.cpp:377`；实现 615–680）。同属虚标：“Optimal (near)” ($\mu < 10^3 \cdot \text{tol_gap}$ 且目标稳定 3 步即成功，不问对偶) 与 “stall-accept” ($|\mu| < 10^{-3}$ 即成功)（证据：533–544, 567–579）。另：无 $Q \succeq 0$ 凸性检查，非凸 QP 无任何检测（证据：全文件无 PSD 判定）。

P7. LP-IPM 三个死选项 + 头文件虚标 `IPMLPOptions` (`ipm_lp_solver.hpp:30--48`) 中：

- **crossover**: 头文件声称 “Crossover to basic feasible solution for MILP warm-starting”；全 cpp 中零消费、零实现 (`grep` 无 `opt_.crossover`)；
- **max_correctors**: 声称 “Max Gondzio centering correctors per iteration (默认 3)”；cpp 零消费——实际每迭代单校正（另有 `skip` 启发），无多重 Gondzio 修正；
- **presolve**: 声称 “Enable presolve (fixed vars, singleton rows, etc.)”；cpp 零消费——不存在内部 presolve（仅有可选的 HiGHS presolve，且由另一选项 `use_highs_presolve` 控制）。

P8. use_inertia_correction 选项未接线 `ipm_solver.hpp:18--19` 注释明确 “Filter must be combined with `use_inertia_correction=true`”，第 49 行声明默认 `true`；全库(`src/include/tests/docs`)仅此两处出现，零消费点。惯性校正实际上无条件进行(`factor_kkt_inertia_corrected_sparse`)，该开关是占位符。

P9. NLP IPM 完全无视 NLP SolverOptions `NLPModel::solver_options`(8 个字段：`max_iterations`、`tolerance`、`acceptable_tolerance`、`dual_/constraint_/complementarity_tolerance` 等, `problem_types.h`) 在 native IPM 路径零消费 (`grep solver_options` 于 `kernel/ipm/`、`solver/native/` 无结果)。用户经通用接口设置的容差静默无效。

P10. 恢复触发的死分支 触发条件之一匹配状态串 “Filter: line-search step too small”（证据：`ipm_solver.cpp:1879`），但全库无任何位置产生该串 (`grep` 仅此一处引用)——该触发分支永不成立（其余三个触发串真实可达，故恢复机制本身仍可触发）。

P11. 无不可行诊断 + 失败静默委托外部 Ipopt NLP-IPM 全文件无 infeasibility 证书 (`grep` 无对应输出路径)；失败链为 `restoration` → `Merit` → `Ipopt` 暖态精化 → `try_ipopt_fallback` 整体重解（证据：`ipm_solver.cpp:1749--1821, 1996--2108`）。兜底结果标注为 `NativeIPM[IpoptFallback]`（诚实），但意味着：任何以 `NativeIPM` 名义产出的基准结果，若走了兜底，实际是 `Ipopt` 解的；中间档 `NativeIPM[IpoptWarmStart]` 也是 `Ipopt` 提供迭代点、native 仅做认证。

P12. Merit 路径互补容差被静默放宽 `comp_tol = max(tol_complementarity, 10tol_primal)`：用户设 `tol_primal=10-6` 时互补容差被强制放宽到 `10-5`，无论 `tol_complementarity` 设多小（证据：`ipm_solver.cpp:2223`）。

4.2 中等（语义偏离/鲁棒性）

- **SOC 仅覆盖等式**: 触发条件含 `r_eq.size()>0`; 纯不等式问题永不触发(证据: `ipm_solver.cpp:1564--1565`)。
- **restoration 仅覆盖等式**: 纯不等式问题无恢复阶段（门控 `prob.g`）（证据: 1885）。
- **对偶步长随原始 α 同步回退**: $\alpha_{dual} = \min(\alpha, \alpha_{dual}^{\max})$ 在回退循环内逐次缩小（证据: 1461）；Ipopt 中 α_{dual} 由 fraction-to-boundary 一次确定，不在回退中收缩。
- **μ 下调时清空 filter**（证据: 1731）：丢失历史禁止域（Ipopt 不清空）。
- **“惯性”并非分解惯性**: 无后端提供负主元计数（MUMPS 的 `negative_eigenvalues()` 存在但因 MUMPS 非默认而不可及）；证书为 $Z^T W Z$ 稠密广义特征解（nullity ≤ 128 门控）或更强的全空间 $W + \delta I \succ 0$ （Eigen SimplicialLDLT 无主元 LDL^T ，对不定矩阵的 D 符号判定理论上不可靠；且 $H \succ 0$ 远强于惯性 $(n, m, 0)$ ，等于对非凸问题逐步凸化）（证据: `kkt_system.cpp:384--407, 562--772`；后端能力 `linear_solver.cpp:404--421`）。
- **增广 Newton 路径自承无惯性证书**: 注释明言 “no formal inertia certificate on this path yet”（证据: `ipm_solver.cpp:686--687`）。
- **Merit 路径无 restoration、无 $\bar{\mu}$ 调度、盲正则**: 停滞后 recenter 重置松弛/乘子，属非标准重启（证据: 2290–2294, 470–480）。
- **max_outer=60 硬编码(1092)**; LCQP 将 max_iter 静默截断到 500(`lcqp_solver.cpp:472`)。

4.3 轻微

- **nonlinear_inequality_multipliers** 对维度不一致静默取 min 截断(证据: `ipm_solver.cpp:67--74`)。
- 缩放模型未拷贝 `solver_options` (P9 的连带项)（证据: `ipm_scaling.cpp:79--98`）。
- τ 恒为 `alpha_max=0.95` (`max(0.5, min(0.95, 1 - 0.01 $\bar{\mu}$ ))`)，无 Ipopt 的渐进 $\tau \rightarrow 1$ 调度（证据: 1353–1354）。
- 测试中的“边界即无限”阈值不一致: NLP 用 10^{19} (27 行) 而 `VariableMeta` 默认界为 $\pm 10^{20}$ （语义上兼容，但属脆弱约定）。

5 数据结构与算法效率评估（通用 NLP 求解器视角）

5.1 每迭代成本链（Filter 驱动，默认路径）

一次主迭代的实际开销（ n 变量、 m_{eq} 等式、 m 合并不等式、nnz 为相应 Jacobian/Hessian 非零数）：

1. **导数评估**: `grad+g+jac_g+h+jac_h` 各一次 (`evaluate_nlp_state`)，Hessian 一次——正常；

2. **凝聚矩阵** $W = H + J_h^\top (M/S) J_h$: 稀疏转置乘, fill 最坏 $O(m \cdot \text{nnz}(J_h))$; 增广路径免此积——正常折中;
3. **KKT 装配**: scatter map $O(\text{nnz})$ 复用, 符号分解缓存——**优** (证据: `kkt_system.cpp:93--242`);
4. **分解**: 默认非对称 LU 作用于对称不定 K ——fill/flops 约为对称不定 $\text{LDL}^\top$ 的 2 倍, 且无惯性输出; 这是与 Ipopt(MA57/Pardiso) 最大的单项效率与能力差距 (证据: `linear_solver.cpp:404--421`);
5. **惯性“认证”**: $\text{nullity} \leq 128$ 时构建显式零空间基 (每自由列一次 $m_{eq} \times m_{eq}$ 回代) + 稠密广义特征解 $O(\text{nullity}^3)$; 否则全空间 SimplicialLDLT——小问题就是创新点, 大问题退化为强凸化;
6. **线搜索**: 每次回退重算全部 **Jacobian** (证据: `evaluate_trial_point → evaluate_nlp_state, ipm_solver.cpp:849--884`)。Ipopt 在线搜索中只评 f 与约束函数值 (g, h) , Jacobian 每迭代只评一次。对 Jacobian 昂贵的模型 (自动微分/有限差分成本 $O(n)$ 每列), 回退 10 次的成本可达整步 Newton 的数倍——**本审计认定的第一效率 red flag**;
7. **IR**: ≤ 2 步迭代精化, 残差用 `kkt_orig` (每次分解整矩阵拷贝 $O(\text{nnz})$) ——可接受 (证据: `kkt_system.cpp:209,266--277`)。

5.2 结构性 red flags

- **界拆双行**: 全界问题 $m = m_{nl} + 2n$; 增广 KKT 维数 $n + m_{eq} + m$ 对界主导问题约为 $3n$, 而 Ipopt 界处理不增维。凝聚路径下 $J_h^\top (M/S) J_h$ 虽只贡献对角, 但 s, μ 存储与逐分量运算翻倍。
- **Merit 路径 reduced-KKT 重复分解**: `solve_kkt_reduced_sparse` 每次调用新建分解器并对 W 重新 `analyze+factorize` (证据: `kkt_system.cpp:283--354`); 每迭代 `affine + corrector` + ≤ 2 Gondzio 共 ≤ 4 次全量分解 (同一 W) (证据: `ipm_solver.cpp:2317,2357,2408`)。非 reduced 路径 ($m_{eq} > 8$) 才有分解复用。
- **失败路径的乘法成本**: 最坏情况 = Filter 全预算 + restoration NLP (≤ 200 迭代) + 原问题重解 (全预算) + Merit (全预算) + Ipopt——可达单次的约 5 倍挂钟 (证据: 1877–2108)。
- **QN 死计算 (P2)**: $n \leq 2000$ 时每迭代白白更新对角 + 块。
- **无列缩放 (P/§2.8) 与绝对容差终止**: 对尺度不良问题, 既少一层预处理又多一分误判风险。

5.3 各内核效率小结

模块	效率评价
LP-IPM	真实 Mehrotra; 正规方程稀疏/带状/稠密三后端 + 稠密列防护 + μ 自适应正则; 节点 LP 结构缓存 (scatter/符号分解/CSR 复用) 是真工程。缺陷: 无多重 Gondzio (选项死)、无 crossover (选项死)、绝对容差、单校正。
LCQP-IPM	符号分解复用 + 对角增量更新, 方向却是单步中心法; 固定 $\rho=10^{-8}$ 、无 IR; 两个无对偶约束的“收敛”出口使迭代预算语义失真。
锥 IPM	全套真实实现 : NT 缩放 (SOC 闭式/SDP SVD)、ECOS 式相对终止、不可行证书、IR、CHOLMOD→MUMPS 回退、Schur 装配 OpenMP 并行、弦分解通道; 每迭代成本 $O(\text{nnz}(K) + \sum p_j^3)$, 与 CVXOPT/ECOS 同阶。
KKT/线代	scatter-map 装配与符号分解缓存是亮点; 默认非对称 LU 解对称不定系统是最大短板; 惯性缺真实来源。

6 测试覆盖与文档一致性

- `tests/test_ipm_solver.cpp` (10 个用例/小节) 与 `tests/test_conic_ipm.cpp` (17 个) 断言是**真实的** (目标值近似、KKT 因子复用计数、证书存在性), 但全部为小规模良性算例; 无大规模/非凸/病态/无 Hessian 回调的用例——P1-P6 所列问题均不在测试射程内。
- `docs/conic_sdp.md` 与锥 IPM 实现一致性高 (NT/Mehrotra/后端回退均属实)。
- `docs/code_quality_evaluation.md` 已自知两点 (LP IPM 曾缺不可行检测、NLP IPM 无原生不可行证书), 与本报告 P11 一致; 但该文档未覆盖 P1-P8。
- `ipm_lp_solver.hpp` 与 `lcqp_solver.cpp` 头注释存在虚标 (P6、P7); `ipm_solver.hpp` 的 `use_inertia_correction` 注释 (P8) 同理。

7 结论与整改优先级

真实性总评: 锥 IPM 与 LP-IPM 是真实、可用、工程水准中上的实现; NLP-IPM 的框架部件 (filter 数据结构、restoration、缩放、KKT 装配缓存、SOC 骨架) 是真实的, 但**收敛语义被 P1/P4/P5 破坏**、**Hessian 供应链被 P2/P3 削弱**、**兜底链 P11 使“native”结果的真实性依赖外部 Ipopt**; LCQP 的头注释名实不符 (P6)。用户关于“大量占位符”的判断在**选项层**完全成立 (P7-P10: 声明即文档、消费为零), 在**算法层**部分成立 (P4-P6: 名实不符), 但不适用于锥/LP 内核主体。

整改优先级:

1. P1: acceptable 级别必须同时约束对偶与互补 (或删除该后门并显式失败);
2. P11: benchmark/报表中区分 native 收敛与 Ipopt 兜底; 补不可行证书;
3. P4/P5: 移除 direct-accept 旁路 (或改文档为“filter/merit 混合”), 修复恒真子句;
4. P2/P3: QN 路径要么真正消费对角 + 块更新, 要么删除; 对仅提供 `hess` 的非线性约束问题显式警告;

5. P6/P7/P8/P9/P10: 删除或实现死选项, 修正头文件虚标;
6. 效率: 线搜索试点只评函数值; Merit 路径分解复用; 接入对称不定后端 (MUMPS 设为可选默认) 并用真实惯性替代证书启发。

A 选项接线普查表

统计口径: 全库 grep 字段名, 人工判定消费点是否影响控制流 (声明处/头文件注释不计)。

选项	状态	消费点/说明
IPMOptions (ipm_solver.hpp)		
max_iter/tol_primal/tol_dual/tol_complementarity	接线	主循环与收敛判据
tol_accept	接线 (负面)	P1 后门, ipm_solver.cpp
alpha_max	接线	fraction-to-boundary
reduced_kkt_max_eq	接线	ipm_solver.cpp:2318 等
qn_sparse_block_size/qn_max_blocks	半接线	进入 QN 态, 但 $n \leq 2000$
verbose	接线	日志
equality_dual_start/inequality_dual_start/slack_start	接线	1052–1066
globalization	接线	1856
use_inertia_correction	未接线	P8, 全库零消费
use_augmented_newton	接线	1222
use_second_order_correction/use_restoration_phase	接线	1564,1877
scale_problem/scaling_g_max	接线	1863–1872
restoration_zeta	接线	1892
mu_init/mu_min/kappa_mu/theta_mu/kappa_epsilon	接线	1042–1044,1119,1721–1724
filter_gamma_theta/filter_gamma_phi	接线	1490–1492 等
filter_s_theta/filter_s_phi/filter_delta/filter_eta_phi/filter_alpha_min	接线	1494–1504,1460
filter_theta_min_scale	只读不用	P5: 进入恒真子句
IPMLPOptions (ipm_lp_solver.hpp)		
max_iter/time_limit_sec/tol_*	接线	主循环
ruiz_rounds	接线	395–405,520
max_correctors	未接线	P7
presolve	未接线	P7
crossover	未接线	P7
verbose	接线	1737
use_highs_presolve	接线	419
NLPSolverOptions (problem_types.hpp:220--233, 8 字段)		
全部 8 字段	native 未接线	P9 (外部 Ipopt 适配器消费)
ConicIPMOptions (conic_ipm_solver.hpp)		
全部字段 (含 chordal_*)	接线	conic_ipm_solver.cpp:86